

Universidad de Sevilla

Escuela Técnica Superior de Ingeniería Informática

Testing Report



Grado en Ingeniería Informática – Ingeniería del Software

DISEÑO Y PRUEBAS II

Curso 2024 – 2025

Fecha	Versión
14/10/2025	v3r0

Grupo: C3.018	
Autores	
Pérez Garrido, Rubén	rubpergar@alum.us.es

Repositorio GITHUB: <https://github.com/rubpergar/Acme-ANS-C3>

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

Índice de contenido

1. Tabla de versiones	3
2. Resumen ejecutivo	4
3. Introducción	5
4. Functional testing	6
5. Performance testing	15
6. Comparativa máquinas	18
7. Conclusiones	22
8. Bibliografía	23

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

1. Tabla de versiones

Fecha	Versión	Descripción
19/05/2025	v1r0	Creación del documento y redacción de los puntos: Resumen ejecutivo, Introducción y parte del Functional Testing.
26/05/2025	v1r1	Corrección de contenido anterior y compleción de los puntos restantes.
03/07/2025	v2r0	Actualización de contenidos acorde a los cambios realizados para la entrega C2
14/10/2025	v3r0	Actualización de contenidos acorde a los cambios realizados para la entrega C3

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

2. Resumen ejecutivo

Para este documento de testing se han desarrollado y explicado cada una de las implementaciones presentes en los requisitos obligatorios de la entrega C3.

Se han evaluado tanto el rendimiento funcional de todas y cada una de las funciones solicitadas, en este caso, las relativas a “flight assignment” y a “activity log”. Para ello se siguió la metodología proporcionada en “S01 - Formal testing” y “S02 - Performance testing”.

El sistema muestra un comportamiento robusto en términos de funcionalidad. En términos de rendimiento, las dos computadoras en las que se ha probado el sistema, tras el estudio realizado, nos confirma que la primera es más poderosa que la segunda.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

3. Introducción

Este documento se divide en dos secciones distintas:

1. Functional testing: se presentarán los casos de prueba implementados, agrupados por funcionalidad. Por cada uno se dará una descripción y una indicación de cuán efectivo es detectando errores. Para la efectividad, se usará el coverage del código para comprobar que se han probado todas las decisiones posibles durante la ejecución del programa y así evitar la existencia de errores.
2. Performance testing: se proporcionarán los gráficos adecuados y un intervalo de confianza del 95% para el tiempo tomado para las solicitudes en las pruebas en dos ordenadores distintos. Además, tras las pruebas en los diferentes ordenadores, se indicará cuál de estos es el más potente y ofrece mejor rendimiento.

4. Functional testing

Casos de prueba relativos a asignaciones de vuelos (Flight Assignments):

acme.features.authenticated.flightCrewMember.flightAssignment	99,7 %	2.347	8	2.355
> FlightAssignmentPublishService.java	99,5 %	658	3	661
> FlightAssignmentUpdateService.java	99,6 %	666	3	669
> FlightAssignmentCreateService.java	99,8 %	559	1	560
> FlightAssignmentShowService.java	99,6 %	251	1	252
> FlightAssignmentController.java	100,0 %	42	0	42
> FlightAssignmentDeleteService.java	100,0 %	67	0	67
> FlightAssignmentListCompletedService.java	100,0 %	52	0	52
> FlightAssignmentListUncompletedService.java	100,0 %	52	0	52

- Create
 - Descripción: Se valida la creación de una asignación mediante la prueba de las restricciones de todos los campos del formulario de creación de una asignación de vuelo con valores relativos a casos positivos, negativos y de hacking. Se valida que los campos de solo lectura no permitan la intrusión de valores a través de las herramientas de desarrollador del navegador y que al hacer Post Hacking de los campos Status, Duty y Leg, salte un error 500. Se valida que un usuario no registrado como miembro (flight crew member) no tenga acceso a esta funcionalidad y que un miembro no disponible no pueda ser asignado para un flight assignment.
 - Coverage: 99,8%
 - Efectividad: Entendemos que una cobertura alrededor del 95% es un valor alto de efectividad y por tanto, al obtener un 99,8% de cobertura, se puede afirmar que el nivel de efectividad es el más alto que se puede obtener, pues todas las líneas de código de la funcionalidad son probadas y se ha verificado que no introduzcan ningún bug o error. Sin embargo, se ha detectado que hay un caso en el que no todas las ramas posibles han sido probadas:

```
if (super.getRequest().getPrincipal().hasRealmOfType(FlightCrewMember.class)) {
```

- Pese a haber validado el acceso a esta funcionalidad tanto con usuarios con el rol FlightCrewMember como con otros usuarios sin rol o rol distinto al permitido, esta línea continúa saliendo en amarillo señalando que faltan ramas por probar.

```
if (duty != null && !"0".equals(duty)) {
```

- En este caso, la única rama que faltaría por probar es aquella en la que el valor de duty sea null y 0 simultáneamente, lo que no es posible, por ello, queda indicado como que una rama falta por ser testeada.

```
if (status != null && !"0".equals(status)) {
```

- Igual que el caso anterior pero con el campo status.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

```
if (leg != null && leg != 0) {
```

- Similar a los 2 casos anteriores pero con el campo leg.

```
List<Leg> availableLegs = allAvailableLegs.stream()
    .filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == airlineId).collect(Collectors.toList());
```

- Esta lista de legs disponibles aparece en amarillo en la cobertura debido a que en los casos de pruebas realizados, no se ha hecho falta filtrar aquellos legs pertenecientes a la misma aerolínea que el usuario, sin embargo, la funcionalidad de implementación es la esperada, ya que en el unbind se emplea la misma estructura y funciona de la manera que debe.
- Pese a estas anotaciones, la funcionalidad de creación de asignaciones de vuelo funciona de la manera esperada y solicitada por el cliente.
- Delete
 - Descripción: Se prueba la eliminación de una asignación de vuelo. Se valida que solo el propietario de la asignación de vuelo (sin publicar) puede eliminarla, eliminando así todos los registros de actividades vinculados a dicha asignación (si tiene). Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
 - Coverage: 100%
 - Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error.
- List completed
 - Descripción: Se prueba el listado de asignaciones de vuelo con tramos completados, es decir, que la fecha de finalización del tramo es anterior a la fecha actual. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad y que un miembro solo pueda ver el listado de sus flight assignments con tramos completados.
 - Coverage: 100%
 - Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error.
- List Uncompleted
 - Descripción: Se prueba el listado de asignaciones de vuelo con tramos incompletos, es decir, que la fecha de finalización del tramo es posterior a la fecha actual. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad y que un miembro solo pueda ver el listado de sus flight assignments con tramos no completados.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error.
- Publish
 - Descripción: Se prueba la publicación de una asignación de vuelo. Se valida que solo el propietario de la asignación de vuelo puede publicarla. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad y que no se pueda publicar un flight assignment con un miembro no disponible. Además, se valida la actualización de los datos de la asignación mediante la prueba de las restricciones de todos los campos del formulario de actualización con valores relativos a casos positivos, negativos y de hacking. Se valida que los campos de solo lectura no permitan la intrusión de valores a través de las herramientas de desarrollador del navegador. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad y que un miembro no disponible no pueda ser asignado para un flight assignment.
 - Coverage: 99,5%
 - Efectividad: Entendemos que una cobertura alrededor del 95% es un valor alto de efectividad y por tanto, al obtener un 99,5% de cobertura, se puede afirmar que el nivel de efectividad es el más alto que se puede obtener, pues todas las líneas de código de la funcionalidad son probadas y se ha verificado que no introduzcan ningún bug o error. Sin embargo, se ha detectado que hay un caso en el que no todas las ramas posibles han sido probadas:

```
if (fa != null) {
    boolean isOwner = super.getRequest().getPrincipal().hasRealm(fa.getFlightCrewMember());
```

- Para este caso de authorise, pese a que se probó la actualización usando identificadores inválidos, se sigue mostrando esta línea como incompleta. Sin embargo, en el caso de que la asignación de vuelo sea null, saldría directamente del bloque if, causando una desautorización al usuario.

```
if (isOwner && fa.isDraftMode())
    status = this.validateRelatedEntities();
```

- En este caso, también se han probado todas las posibilidades, tanto acceso a la funcionalidad por parte del propietario de la asignación de vuelo (publicada y sin publicar), como por parte de un usuario no propietario de la asignación de vuelo (publicada y sin publicar), sin embargo, se sigue mostrando esta línea en amarillo.

```
if (duty != null && !"0".equals(duty)) {
    if (status != null && !"0".equals(status)) {
```


- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

```
int assignmentId = super.getRequest().getData("id", int.class);  
if (assignmentId != 0) {
```

- Para este caso, pese a haber probado con asignaciones de vuelo con identificadores con valor válido, 0 e inválido, esta línea sigue mostrándose en amarillo.

```
if (original != null && original.getLeg() != null) {
```

- Aquí se valida si el leg presente en el formulario era el que se encontraba inicialmente para evitar problemas si la asignación de vuelo fue creada cuando el leg era futuro y ahora ha cambiado a pasado. Para este caso, la única rama que no ha sido probada es en la que la asignación de vuelo original es nula, sin embargo, en caso de serlo, saldría del bloque if y continuaría la ejecución de manera esperada.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == airlineId).collect(Collectors.toList());
```

- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

```
if (originalLeg != null && assigned.getId() == originalLeg.getId())
```

- Aquí se valida si la leg asignada es la misma que la que había anteriormente para evitar, nuevamente, problemas si la leg ha pasado de futuro a pasada. Para este caso, la única rama que no ha sido probada es en la que la asignación de vuelo original es nula y la actual también, sin embargo, en caso de suceder esta combinación, saldría del bloque if y se procesaría el siguiente leg de la lista de manera esperada.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == flightCrewMemberAirlineId).collect(Collectors.toList());
```

- En este caso faltaría pasar por la rama que genera una lista vacía en el filtrado. Esta situación simplemente indicaría que no se tiene ninguna asignación publicada para ese leg que ya contenga un piloto o copiloto.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == flightCrewMemberAirlineId).collect(Collectors.toList());
```

- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

- Show

- Descripción: Se prueba la visualización de los detalles de una asignación de vuelo. Se valida que un usuario registrado como miembro pueda visualizar los detalles de la asignación siempre y cuando esta se encuentre publicada. El miembro al que le corresponde la asignación de vuelo siempre podrá ver los detalles independientemente de si está publicada o no. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad independientemente de si la asignación está publicada o no.
- Coverage: 99,6%
- Efectividad: Entendemos que una cobertura alrededor del 95% es un valor alto

de efectividad y por tanto, al obtener un 99,6% de cobertura, se puede afirmar que el nivel de efectividad es el más alto que se puede obtener, pues todas las líneas de código de la funcionalidad son probadas y se ha verificado que no introduzcan ningún bug o error. Sin embargo, se ha detectado que hay un caso en el que no todas las ramas posibles han sido probadas:

```
if (super.getRequest().hasData("id")) {
```

- Pese a haber hecho todas las pruebas necesarias mostrando asignaciones de vuelos con identificadores válidos e inválidos, esta línea se sigue mostrando como incompleta.

```
if (flightAssignment != null) {
```

- Misma situación que en la funcionalidad de publicación de asignaciones de vuelo.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().get airlineNumber().get airline().getId() == flightCrewMemberAirlineId).collect(Collectors.toList());
```

- Misma situación que en la funcionalidad de publicación de asignaciones de vuelo.

- Update

- Descripción: Se valida la actualización de los datos de una asignación mediante la prueba de las restricciones de todos los campos del formulario de actualización con valores relativos a casos positivos, negativos y de hacking. Se valida que los campos de solo lectura no permitan la intrusión de valores a través de las herramientas de desarrollador del navegador. Se valida que solo el propietario de la asignación de vuelo (sin publicar) puede actualizarla. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad y que un miembro no disponible no pueda ser asignado para un flight assignment.
- Coverage: 99,6%
- Efectividad: Entendemos que una cobertura alrededor del 95% es un valor alto de efectividad y por tanto, al obtener un 99,6% de cobertura, se puede afirmar que el nivel de efectividad es el más alto que se puede obtener, pues todas las líneas de código de la funcionalidad son probadas y se ha verificado que no introduzcan ningún bug o error. Sin embargo, se ha detectado que hay un caso en el que no todas las ramas posibles han sido probadas:

```
if (fa != null) {  
    boolean isOwner = super.getRequest().getPrincipal().hasRealm(fa.getFlightCrewMember());
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.



```
if (isOwner && fa.isDraftMode())  
    status = this.validateRelatedEntities();
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

```
if (duty != null && !"0".equals(duty)) {  
if (status != null && !"0".equals(status)) {
```

- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

```
int assignmentId = super.getRequest().getData("id", int.class);  
if (assignmentId != 0) {
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

```
if (original != null && original.getLeg() != null) {
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == airlineId).collect(Collectors.toList());
```

- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

```
if (originalLeg != null && assigned.getId() == originalLeg.getId())
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == flightCrewMemberAirlineId).collect(Collectors.toList());
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

```
List<Leg> availableLegs = allAvailableLegs.stream().filter(l -> l.getFlight().getAirlineManager().getAirline().getId() == flightCrewMemberAirlineId).collect(Collectors.toList());
```

- Mismo caso que en la funcionalidad de creación de asignación de vuelo.

Casos de prueba relativos a registros de actividad (Activity Logs):

acme.features.authenticated.flightCrewMember.activityLog	100,0 %	951	0	951
> ActivityLogController.java	100,0 %	35	0	35
> ActivityLogCreateService.java	100,0 %	198	0	198
> ActivityLogDeleteService.java	100,0 %	63	0	63
> ActivityLogListService.java	100,0 %	119	0	119
> ActivityLogPublishService.java	100,0 %	209	0	209
> ActivityLogShowService.java	100,0 %	134	0	134
> ActivityLogUpdateService.java	100,0 %	193	0	193

- Create

- Descripción: Se valida la creación de un registro de actividad mediante la prueba de las restricciones de todos los campos del formulario de creación con valores relativos a casos positivos, negativos y de hacking. Se valida que los campos de solo lectura no permitan la intrusión de valores a través de las herramientas de desarrollador del navegador. Se valida que un miembro que no pertenece a la asignación de vuelo, no pueda introducir registros de actividad para dicha asignación. Se valida que un registro de actividad no se puede crear si la asignación de vuelo a la que se pertenece se encuentra en un tramo sin completar. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

```
if (super.getRequest().hasData("masterId", int.class)) {
```

- Mismo caso que en la funcionalidad de mostrar asignación de vuelo.

```
FlightAssignment fa = this.repository.findFlightAssignmentById(masterId);
if (fa != null)
```

- Mismo caso que en la funcionalidad de publicación de asignación de vuelo.

- Delete

- Descripción: Se prueba la eliminación de un registro de actividad. Se valida que solo el propietario del registro (sin publicar) puede eliminarlo. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

```
FlightAssignment flightAssignment = this.repository.findFlightAssignmentByActivityLogId(activityLogId);
if (flightAssignment != null)
```

- Mismo caso que en la funcionalidad de crear registro de actividad.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

- List

- Descripción: Se prueba el listado de registros de actividad. Se valida que solo el propietario de la asignación de vuelo (sin publicar) puede acceder a este listado. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

```
if (super.getRequest().hasData("masterId", int.class)) {
    FlightAssignment fa = this.repository.findFlightAssignmentById(masterId);
    if (fa != null)
```

- Mismo caso que en la funcionalidad de crear registro de actividad.

- Publish

- Descripción: Se prueba la publicación de un registro de actividad. Se valida que solo el propietario del registro puede publicarla. Se valida que un registro no pueda ser publicado si la asignación de vuelo a la que pertenece no se encuentra publicada. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

```
if (flightAssignment != null) {
    boolean isOwner = super.getRequest().getPrincipal().hasRealm(flightAssignment.getFlightCrewMember());
```

- Mismo caso que en la funcionalidad de crear registro de actividad.

- Show

- Descripción: Se prueba la visualización de los detalles de un registro de actividad. Se valida que un usuario registrado como miembro pueda visualizar los detalles del registro siempre y cuando este se encuentre publicado. El miembro al que le corresponde el registro de vuelo siempre podrá ver los detalles independientemente de si está publicada o no. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad independientemente de si la asignación está publicada o no.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

```
if (super.getRequest().hasData("id", int.class)) {
    ActivityLog activityLog = this.repository.getActivityLogById(id);
    if (activityLog != null) {
```

- Caso muy similar al de la funcionalidad de creación registro de actividad

- Update

- Descripción: Se valida la actualización de los datos de un registro de actividad mediante la prueba de las restricciones de todos los campos del formulario de actualización con valores relativos a casos positivos, negativos y de hacking. Se valida que los campos de solo lectura no permitan la intrusión de valores a través de las herramientas de desarrollador del navegador. Se valida que solo el propietario de la asignación de vuelo (sin publicar) puede actualizarlo. Se valida que un usuario no registrado como miembro no tenga acceso a esta funcionalidad.
- Coverage: 100%
- Efectividad: Máxima, ya que se han probado todas las líneas de código de la funcionalidad y se ha verificado que no introduzcan ningún bug o error. Sin embargo, hay ciertos detalles a destacar:

```
if (flightAssignment != null) {
    boolean isOwner = super.getRequest().getPrincipal().hasRealm(flightAssignment.getFlightCrewMember());
```

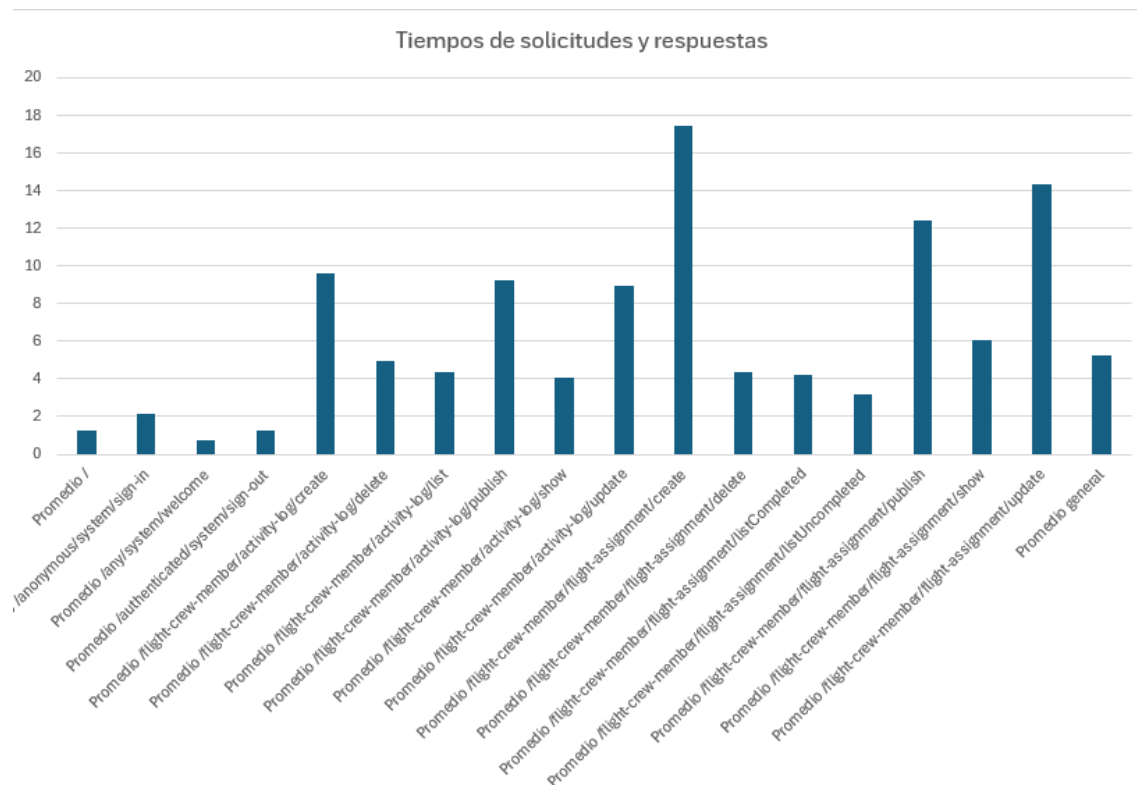
- Mismo caso que en la funcionalidad de crear registro de actividad.

En resumen, se han validado el todas las funcionalidades con el objetivo de evitar la presencia de bugs y errores no esperados.

5. Performance testing

Tras realizar el conjunto de tests para las funcionalidades oportunas, se han realizado todos los pasos que se muestran en “S02 - Performance testing”, obteniendo los siguientes resultados:

request-path	time
Promedio /	1,28087667
Promedio /anonymous/system/sign-in	2,10947636
Promedio /any/system/welcome	0,76677472
Promedio /authenticated/system/sign-out	1,28575918
Promedio /flight-crew-member/activity-log/create	9,57611042
Promedio /flight-crew-member/activity-log/delete	4,964525
Promedio /flight-crew-member/activity-log/list	4,36658966
Promedio /flight-crew-member/activity-log/publish	9,24442821
Promedio /flight-crew-member/activity-log/show	4,03637179
Promedio /flight-crew-member/activity-log/update	8,97348158
Promedio /flight-crew-member/flight-assignment/create	17,4467815
Promedio /flight-crew-member/flight-assignment/delete	4,37721
Promedio /flight-crew-member/flight-assignment/listCompleted	4,20576226
Promedio /flight-crew-member/flight-assignment/listUncompleted	3,17795085
Promedio /flight-crew-member/flight-assignment/publish	12,4205673
Promedio /flight-crew-member/flight-assignment/show	6,08387527
Promedio /flight-crew-member/flight-assignment/update	14,320375
Promedio general	5,23749326



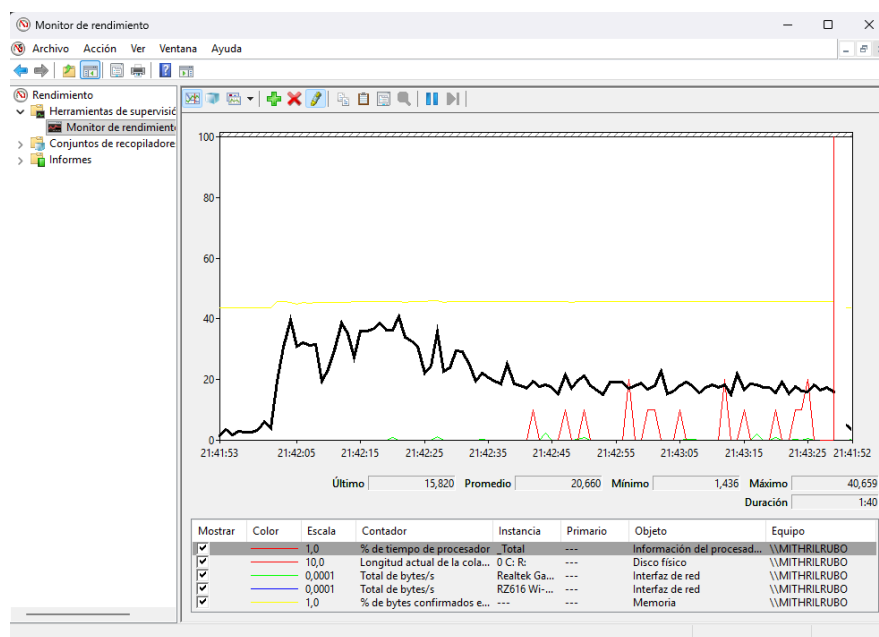
A partir de las dos imágenes anteriores, se observa que el tiempo promedio que tarda el sistema en realizar una petición es de unos 5 ms, es decir, 0,005 segundos. Viendo el gráfico de barras, se puede afirmar de forma visual, que las peticiones que más tiempo tardan en ejecutarse son las relativas a la creación, actualización y publicación de ambas entidades probadas, ya que son las que poseen mayor número de validaciones de campos y anti-hacking, además de poseer consultas que requieren la lectura y tratamiento de un gran volumen de datos de la base de datos.

Posteriormente se realizó un examen a la máquina sobre la que se estaban realizando las pruebas para obtener información acerca de ella y su rendimiento, con el objetivo de así descubrir si esta podría estar limitando la eficiencia del código. Estos fueron los resultados:

Name	Self Time (CPU)	Total Time (CPU)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.unbind ()	0,0 ms (- %)	496 ms (18,6 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentPublishService.authorise ()	0,0 ms (- %)	202 ms (7,6 %)
acme.features.authenticated.flightCrewMember.activityLog.ActivityLogCreateService.validate ()	0,0 ms (- %)	201 ms (7,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.validateRelatedEntities ()	0,0 ms (- %)	200 ms (7,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.authorise ()	0,0 ms (- %)	200 ms (7,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.bind ()	0,0 ms (- %)	200 ms (7,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentListUncompletedService.load ()	0,0 ms (- %)	199 ms (7,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentUpdateService.validate ()	0,0 ms (- %)	189 ms (7,1 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentShowService.authorise ()	0,0 ms (- %)	109 ms (4,1 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentUpdateService.bind ()	0,0 ms (- %)	108 ms (4,1 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.validate ()	0,0 ms (- %)	97,5 ms (3,7 %)
acme.features.authenticated.flightCrewMember.activityLog.ActivityLogShowService.authorise ()	0,0 ms (- %)	95,7 ms (3,6 %)
acme.features.authenticated.flightCrewMember.activityLog.ActivityLogPublishService.validate ()	0,0 ms (- %)	95,0 ms (3,6 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentPublishService.bind ()	0,0 ms (- %)	93,8 ms (3,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentPublishService.unbind ()	0,0 ms (- %)	92,3 ms (3,5 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentCreateService.load ()	0,0 ms (- %)	85,1 ms (3,2 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentShowService.unbind ()	0,0 ms (- %)	0,0 ms (0 %)
acme.features.authenticated.flightCrewMember.flightAssignment.FlightAssignmentUpdateService.authorise ()	0,0 ms (- %)	0,0 ms (0 %)

Respecto al software, en la imagen superior podemos ver los métodos que más tiempo tardan en ejecutarse dentro de las funcionalidades desarrolladas y probadas, siendo con diferencia el unbind de la creación de la asignación de vuelo la que más tiempo consume. Sin embargo, el “Self Time” vale 0,0ms, lo que nos indica que el código que se ha realizado no es el que está consumiendo ese tiempo, sino el método que invoca.

Respecto al hardware, tenemos los siguientes resultados:



A simple vista se observa que no se alcanza el 100% para ninguno de los valores estudiados, por lo que se puede afirmar que el sistema no tiene ningún cuello de botella que pueda comprometer la eficiencia de la aplicación.

Una vez verificados estos puntos y habiendo estudiado el comportamiento del sistema al ejecutar la aplicación, se estudia el añadir índices para las consultas SQL que lo requieran para optimizar la eficiencia del sistema. Tras el estudio de todas las consultas que requieran de algunas de las entidades que he desarrollado (flight assignment, activity log y flight crew member), se observa que el único índice que se requiere es el relativo a esta consulta:

```
@Query("SELECT DISTINCT fa.leg FROM FlightAssignment fa WHERE fa.flightCrewMember.id = :memberId AND fa.leg.id != :legId")
List<Leg> getAllLegsByMemberIdExceptSelfLeg(int memberId, int legId);
```

Como la consulta emplea los identificadores de esas dos entidades conectadas con flight assignment, se crea un índice compuesto con ambas en dicha entidad:

```
@Table(indexes = {
    @Index(columnList = "flight_crew_member_id, leg_id")
})
public class FlightAssignment extends AbstractEntity {
```

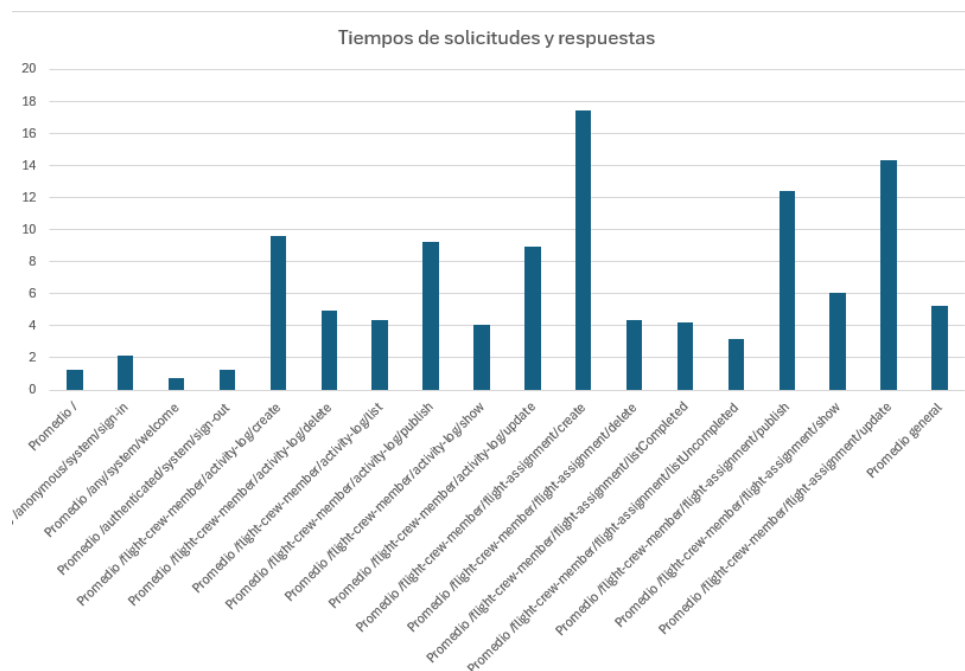
El resto de posibles índices no son necesarios añadirlos ya que el framework los crea automáticamente.

6. Comparativa máquinas

Se comparará ahora el rendimiento de la aplicación en la máquina donde se ha desarrollado el código (Ordenador 1) con el de otra compañera de grupo (Ordenador 2). Estos son los resultados:

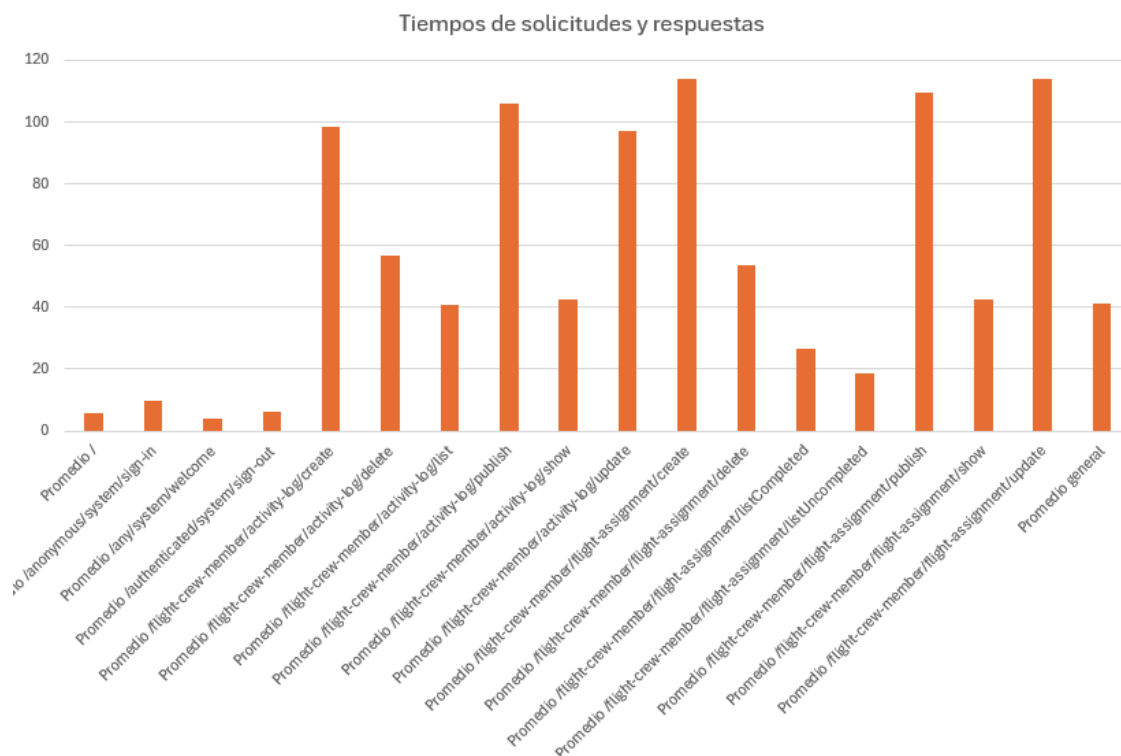
Ordenador 1:

request-path	time
Promedio /	1,28087667
Promedio /anonymous/system/sign-in	2,10947636
Promedio /any/system/welcome	0,76677472
Promedio /authenticated/system/sign-out	1,28575918
Promedio /flight-crew-member/activity-log/create	9,57611042
Promedio /flight-crew-member/activity-log/delete	4,964525
Promedio /flight-crew-member/activity-log/list	4,36658966
Promedio /flight-crew-member/activity-log/publish	9,24442821
Promedio /flight-crew-member/activity-log/show	4,03637179
Promedio /flight-crew-member/activity-log/update	8,97348158
Promedio /flight-crew-member/flight-assignment/create	17,4467815
Promedio /flight-crew-member/flight-assignment/delete	4,37721
Promedio /flight-crew-member/flight-assignment/listCompleted	4,20576226
Promedio /flight-crew-member/flight-assignment/listUncompleted	3,17795085
Promedio /flight-crew-member/flight-assignment/publish	12,4205673
Promedio /flight-crew-member/flight-assignment/show	6,08387527
Promedio /flight-crew-member/flight-assignment/update	14,320375
Promedio general	5,23749326



Ordenador 2:

Promedio /	5,85019667
Promedio /anonymous/system/sign-in	9,86568545
Promedio /any/system/welcome	3,85353876
Promedio /authenticated/system/sign-out	6,08463673
Promedio /flight-crew-member/activity-log/create	98,4099292
Promedio /flight-crew-member/activity-log/delete	56,8243125
Promedio /flight-crew-member/activity-log/list	40,8184828
Promedio /flight-crew-member/activity-log/publish	105,790413
Promedio /flight-crew-member/activity-log/show	42,7210564
Promedio /flight-crew-member/activity-log/update	96,9310763
Promedio /flight-crew-member/flight-assignment/create	114,104766
Promedio /flight-crew-member/flight-assignment/delete	53,49676
Promedio /flight-crew-member/flight-assignment/listCompleted	26,8015906
Promedio /flight-crew-member/flight-assignment/listUncompleted	18,4229034
Promedio /flight-crew-member/flight-assignment/publish	109,481541
Promedio /flight-crew-member/flight-assignment/show	42,5389796
Promedio /flight-crew-member/flight-assignment/update	114,104867
Promedio general	41,4084455



	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

A simple vista se observa que el ordenador 1 arroja mejores tiempos de manera general para todas las tareas, siendo casi 13 ms más rápido que el ordenador 2. Continuando con el estudio comparativo tenemos lo siguiente:

<i>Ordenador 1</i>			<i>Ordenador 2</i>		
Media	5,23749326		Media	41,37468379	
Error típico	0,20055371		Error típico	1,541496928	
Mediana	3,29595		Mediana	16,52085	
Moda	0,6274		Moda	2,6804	
Desviación estándar	6,645564382		Desviación estándar	51,07917022	
Varianza de la muestra	44,16352596		Varianza de la muestra	2609,08163	
Curtosis	33,39391748		Curtosis	4,696698724	
Coefficiente de asimetría	3,974444033		Coefficiente de asimetría	1,864113067	
Rango	92,6409		Rango	410,0677	
Mínimo	0,4508		Mínimo	2,5116	
Máximo	93,0917		Máximo	412,5793	
Suma	5750,7676		Suma	45429,4028	
Cuenta	1098		Cuenta	1098	
Nivel de confianza(95,0%)	0,393512218		Nivel de confianza(95,0%)	3,024615578	
Intervalo (ms)	4,843981043	5,63100548	Intervalo (ms)	38,35006821	44,3992994
Intervalo (s)	0,004843981	0,00563101	Intervalo (s)	0,038350068	0,0443993

Con la imagen superior, se reafirma lo expuesto anteriormente, ya que el ordenador 1 posee un intervalo de confianza (4.84, 5.63) inferior al del ordenador 2 (38.35, 44.39), dejando ver que es muy posible que la primera máquina sea más rápida y eficaz a la hora de ejecutar la aplicación. Para confirmar esta hipótesis, utilizaremos el z-test:

	<i>Ordenador 1</i>	<i>Ordenador 2</i>
Media	5,23749326	41,37468379
Varianza (conocida)	44	2609
Observaciones	1098	1098
Diferencia hipotética de las medias	0	
z	-23,24807139	
P(Z<=z) una cola	0	
Valor crítico de z (una cola)	1,644853627	
P(Z<=z) dos colas	0	
Valor crítico de z (dos colas)	1,959963985	

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

El valor p asociado a la prueba Z de dos colas ($P(Z \leq z)$ dos colas) nos permite determinar si existe evidencia estadísticamente significativa para afirmar que uno de los ordenadores presenta un mejor rendimiento que el otro. En este análisis, se ha establecido un nivel de significación (α) de 0,05.

Se pueden presentar las siguientes situaciones:

1. Si el valor p es menor o igual a 0,05, se concluye que existe una diferencia significativa entre los tiempos medios de ejecución de los dos ordenadores, por lo que se puede realizar la comparativa de los promedios de los tiempos.
2. Si el valor p es mayor a 0,05, implica que no se dispone de evidencia suficiente para afirmar que uno de los ordenadores sea más eficiente que el otro ya que las diferencias globales entre los tiempos no son significativas.
3. Si el valor p es próximo a 0,05, se recomienda considerar la ampliación de la muestra para obtener resultados más precisos y robustos ya que los resultados del test no son conclusivos.

En el caso analizado, el valor p obtenido es prácticamente 0, lo que se traduce en una probabilidad extremadamente baja de que la diferencia observada en los tiempos medios de ejecución sea debida al azar.

Por tanto, se puede concluir que existe una diferencia estadísticamente significativa en los rendimientos de los dos ordenadores. Observando los valores medios de los tiempos de ejecución, se concluye que el ordenador 1 es bastante más eficiente, con un tiempo medio de 5.23 ms, frente a los 41.37 ms del ordenador 2.

Esta diferencia es lo suficientemente clara y robusta como para afirmar que, en este contexto de pruebas, el ordenador 1 ofrece un rendimiento superior.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

7. Conclusiones

Tras la elaboración de este documento sobre testing, se ha concluido que esta fase del ciclo de vida de un proyecto es crucial, ya que aporta mucha información útil sobre factores que no se tienen en cuenta o que pueden pasar por alto durante el desarrollo. Validar que todas las funcionalidades desarrolladas operen correctamente y sean revisadas minuciosamente para minimizar errores o bugs, además de asegurar que el rendimiento esté optimizado al máximo, son factores fundamentales de cara al cliente. Un sistema bien probado permite que el usuario final lo utilice de manera rápida e intuitiva, evitando problemas que puedan afectar negativamente su experiencia. Además, un proceso de testing riguroso contribuye a la satisfacción del cliente y a la reputación del producto, garantizando que las expectativas de calidad y eficiencia sean cumplidas de manera consistente.

	DISEÑO Y PRUEBAS II Testing Report - Rubén Pérez Garrido
---	--

8. Bibliografía

- 06 Annexes – Material proporcionado en la asignatura Diseño y Pruebas II por la Universidad de Sevilla.
- L04 - S01 - Formal testing - Material proporcionado en la asignatura Diseño y Pruebas II por la Universidad de Sevilla.
- L04 - S02 - Performance testing - Material proporcionado en la asignatura Diseño y Pruebas II por la Universidad de Sevilla.